



Customer Experience Survey Management Software: A Secure and Scalable Multi-Tenant Architecture

Quoc-Dung Nguyen¹ · Van-Nam Pham¹ · Trang-Nhung Dang¹ · Tam-Nhu Phan¹ · Nguyet-Minh Phan²

Received: 30 April 2025 / Accepted: 20 September 2025
© The Author(s), under exclusive licence to Springer Nature Singapore Pte Ltd. 2025

Abstract

In today's competitive market landscape, effective customer experience management demands advanced survey management tools that can simultaneously support the needs of multiple businesses. This study presents the design of a secure and scalable multi-tenant architecture tailored to address the complexities of managing surveys for multiple business entities. The proposed architecture ensures data privacy and tenant isolation while enabling flexible and efficient survey management across various business contexts. Key advantages, including improved security protocols and operational effectiveness, are explored. In addition, the system's core functionalities are tested and validated within a controlled test environment.

Keywords Multi-tenancy · Survey software · Security · Data Isolation · Scalability

Introduction

Customer experience management is increasingly acknowledged as a key factor for businesses aiming to gain a competitive edge in today's dynamic market environment. For example, Ivanovska et al. [1] emphasize the importance of touchpoint management of telecommunications companies in understanding customer experiences, which can be effectively captured through survey tools. Similarly, the study by

Halvorsrud et al. [2] provides customer journey analysis for empirical investigation of individual service experiences using the data collected from interviews, diary studies, and process tracking from backend systems. A 2021 survey conducted by Deloitte [3] revealed that companies prioritizing customer experience typically achieve 1.5 times higher customer satisfaction compared to those with less customer-centric approaches. However, traditional survey tools often fall short of meeting the operational requirements of multiple entities within a single system, especially in environments where diverse business units operate under the same corporate structure.

Moreover, the role of customer relationship management (CRM) in improving customer satisfaction is well-documented [4, 5]. By utilizing a multi-tenant architecture, survey management applications can facilitate the collection and analysis of customer feedback, allowing organizations to refine their strategies and improve overall customer loyalty.

This paper presents a novel multi-tenant architecture specifically designed to overcome existing operational challenges. By enabling multiple businesses to share a common survey management platform, the architecture ensures robust data segregation and security for each tenant. As noted in the seminal work by Armbrust et al. [6], multi-tenant architectures offer notable advantages, including

✉ Nguyet-Minh Phan
minhpn@sgu.edu.vn
Quoc-Dung Nguyen
dung.nguyen@vlu.edu.vn
Van-Nam Pham
phamnam7892@gmail.com
Trang-Nhung Dang
lamtieu682@gmail.com
Tam-Nhu Phan
nhuphan160200@gmail.com

¹ Faculty of Mechanical, Electrical, and Computer Engineering, Van Lang School of Technology, Van Lang University, Ho Chi Minh City, Vietnam

² Faculty of Information Technology, Sai Gon University, 273 An Duong Vuong, Cho Quan Ward, Ho Chi Minh City, Vietnam

significant cost reductions and enhanced efficiency through shared resource utilization.

The concept of multi-tenancy has long been established and widely utilized across various applications; however, incorporating it into customer experience survey platforms introduces distinct challenges. As outlined in the AWS whitepaper on Software as a Service (SaaS) tenant isolation strategies [7], although many businesses pursue multi-tenant systems to improve customer management, they often face difficulties in preserving data integrity and adhering to strict regulatory requirements. This underscores the urgent need for a more robust multi-tenant framework capable of addressing the intricate demands of contemporary survey management tools.

This paper is an extended version of the work presented at the 11th International Conference on Future Data and Security Engineering (FDSE 2024) [8]. Its primary contribution is the development of a secure and scalable multi-tenant architecture that leverages advanced security mechanisms and scalable infrastructure to effectively support diverse business operations. The approach is specifically designed for organizations that manage multiple customer interaction models within a unified platform, ensuring strict data isolation and protection for each tenant. The paper presents the architecture's implementation and key functionalities, which are evaluated in a controlled test environment. In addition, it discusses the challenges encountered during development and the corresponding solutions applied and explores the potential advantages and limitations of the proposed approach.

The structure of the paper is as follows. Section 2 provides a review of multi-tenant architectures, along with associated security and scalability challenges and strategies. Section 3 introduces our proposed secure and scalable multi-tenant architecture, designed to offer a robust, scalable, and efficient solution for managing customer experience surveys. In addition, we discuss the implementation details and key observations. Lastly, Sect. 4 concludes the paper by summarizing the main findings and proposing directions for future research.

Literature Review

The evolution of SaaS has led to the widespread adoption of multi-tenant architectures, which allow multiple customers (tenants) to share a single application instance while maintaining data isolation and security. This literature review examines the key components of secure and scalable multi-tenant architectures, particularly in the context of customer experience survey management software.

Multi-tenant Architecture

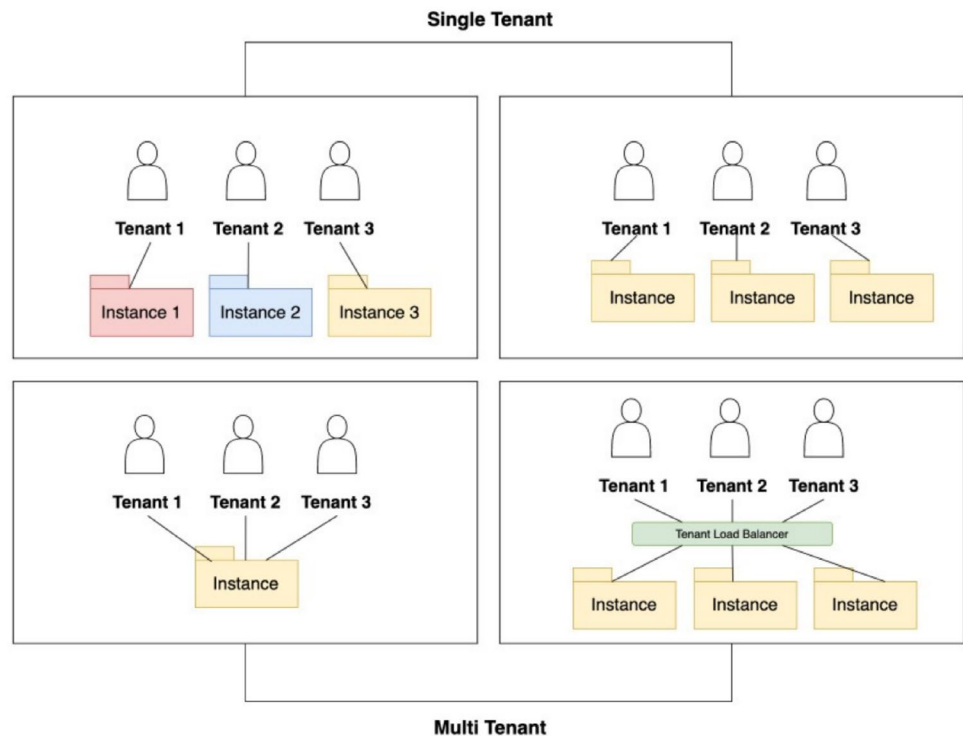
Multi-tenant architecture has emerged as a foundational element in contemporary software development, particularly within cloud-based solutions. As described by Bezemer and Zaidman [9], the core idea centers on the efficient sharing of resources across multiple tenants, leading to reduced operational costs and enhanced efficiency and scalability [10, 11]. This architecture allows for logical separation of resources, ensuring that each tenant's data remains isolated while sharing the underlying infrastructure [12]. Pinto et al. [10] emphasize that multi-tenant architecture not only facilitates cost efficiency but also enables tailored configurations to meet specific business needs, which is crucial for applications like customer experience surveys that require customization for different clients. This approach has been extensively utilized across multiple domains, including customer relationship management systems, enterprise resource planning software, and content management platforms.

The systematic mapping study by Kabbedijk et al. [13] highlights the evolution of multi-tenancy from traditional software deployment models, noting that the demand for personalized software services has driven the development of more sophisticated multi-tenant systems. This evolution is essential for customer experience survey management software, which must adapt to varying client requirements while ensuring data integrity and security.

A multi-tenant architecture is a software architecture in which a single software instance operates on a server and serves multiple distinct user groups, referred to as tenants. While tenants access the same application, they maintain separate privileges and data. This architecture is designed to allocate dedicated resources, such as data, user management, configurations, tenant-specific functionalities, and non-functional properties, to each tenant within the shared environment. Figure 1 illustrates the SaaS maturity model, which outlines the progressive stages of a SaaS application's ability to support multi-tenancy. Each level in the model reflects the degree to which the software can efficiently and securely serve multiple tenants. The levels are defined as follows:

- Level 1: Each tenant operates their separate instance of the software with minimal customization, hosted by the service provider.
- Level 2: Provides greater flexibility, enabling tenants to customize the software to better suit their specific requirements.
- Level 3: Tenants use a shared software application with a common database but with selective customization according to individual tenant needs.

Fig. 1 SaaS Maturity Model



- Level 4: The same software version is used by all tenants, offering full self-service configuration and customization through an intuitive user interface.

Security Challenges and Strategies

Security is a paramount concern in multi-tenant environments, as the shared nature of resources can lead to vulnerabilities if not properly managed. Reddy and Kumar [14] discuss various security challenges inherent in multi-tenancy, including data leakage and unauthorized access. Effective isolation mechanisms are critical to prevent cross-tenant data breaches, which can severely impact customer trust and satisfaction [15]. In multi-tenant architectures, especially those managing sensitive customer experience surveys, ensuring data security and integrity is critically important. Given that such data frequently includes personal customer information, it requires stringent security protocols to protect against breaches and unauthorized access. To address these security challenges, various strategies have been proposed, including the use of tenant isolation and access control mechanisms [15, 16].

Database segregation is an efficient approach to ensuring data security in multi-tenant systems. It involves allocating a separate database to each tenant, creating physical isolation of their data. This strategy guarantees that one tenant's data is fully isolated and inaccessible to others, significantly minimizing the risk of cross-tenant data breaches. As highlighted by Mather et al. [17], this approach not only prevents

unauthorized access between tenants but also facilitates easier compliance with data protection regulations, making it especially effective in environments dealing with sensitive information.

Schema-based isolation offers an effective alternative to database segregation in multi-tenant systems by utilizing a single database with distinct schemas for each tenant. Unlike physical data separation, this approach provides logical isolation, where each schema functions as an independent namespace with its own tables, views, and procedures. This ensures that operations in one schema do not compromise the integrity or security of data in another tenant's schema. Aulbach et al. [18] discuss this data isolation strategy in the context of SaaS platforms, highlighting how schema-based isolation not only preserves data privacy but also improves resource utilization across the system. The study emphasizes that this method simplifies system maintenance and scalability, making it a cost-effective and efficient choice for managing multi-tenant architectures while maintaining robust security and operational efficiency.

Scalability in Multi-tenant Architectures

Scalability is another critical aspect of multi-tenant architectures, particularly for applications that may experience fluctuating demand, such as customer experience surveys during peak times. The proposed multivariable architecture by Samrajesh and Gopalan [19] demonstrates how dynamic scaling of application components can optimize resource

allocation based on load, thereby improving response times and conserving energy. This is particularly beneficial for survey management systems that need to efficiently handle varying volumes of responses.

Cao et al. [20] identify scalability as a key challenge in multi-tenant SaaS development, highlighting the need for flexible deployment strategies that can accommodate growth without compromising performance. The middleware solution presented by Makki et al. [21] further illustrate how decoupling customization concerns from workflow design can enhance scalability, allowing survey management software to adapt to different tenant requirements without incurring significant overhead.

Proposed Secure and Scalable Multi-Tenant Architecture and Implementation

The proposed architecture of the customer experience survey software, named the MiSurvey system, is specifically designed to overcome the challenges of managing customer experience surveys across multiple business entities. This architecture emphasizes security, high scalability, and

operational efficiency while delivering a full suite of features for creating, distributing, and analyzing surveys.

The proposed architecture allows multiple businesses to use the same survey management system while ensuring strict data isolation - an essential requirement for customer experience surveys where data privacy and security are critical. The architecture supports efficient resource sharing among different tenants, which can reduce operational costs and improve scalability, which are key benefits for companies looking to manage surveys across various business units without compromising privacy. It employs database segregation to ensure that the data of one tenant remains inaccessible to others, which is essential in maintaining the confidentiality and integrity of survey responses. Figure 2 illustrates this architecture.

Architecture Overview

The architecture adopts a multi-tenant model, where a single application instance serves multiple tenants (businesses). Each tenant operates independently within a securely isolated environment, enabling both data separation and customization. The application is composed of the following components:

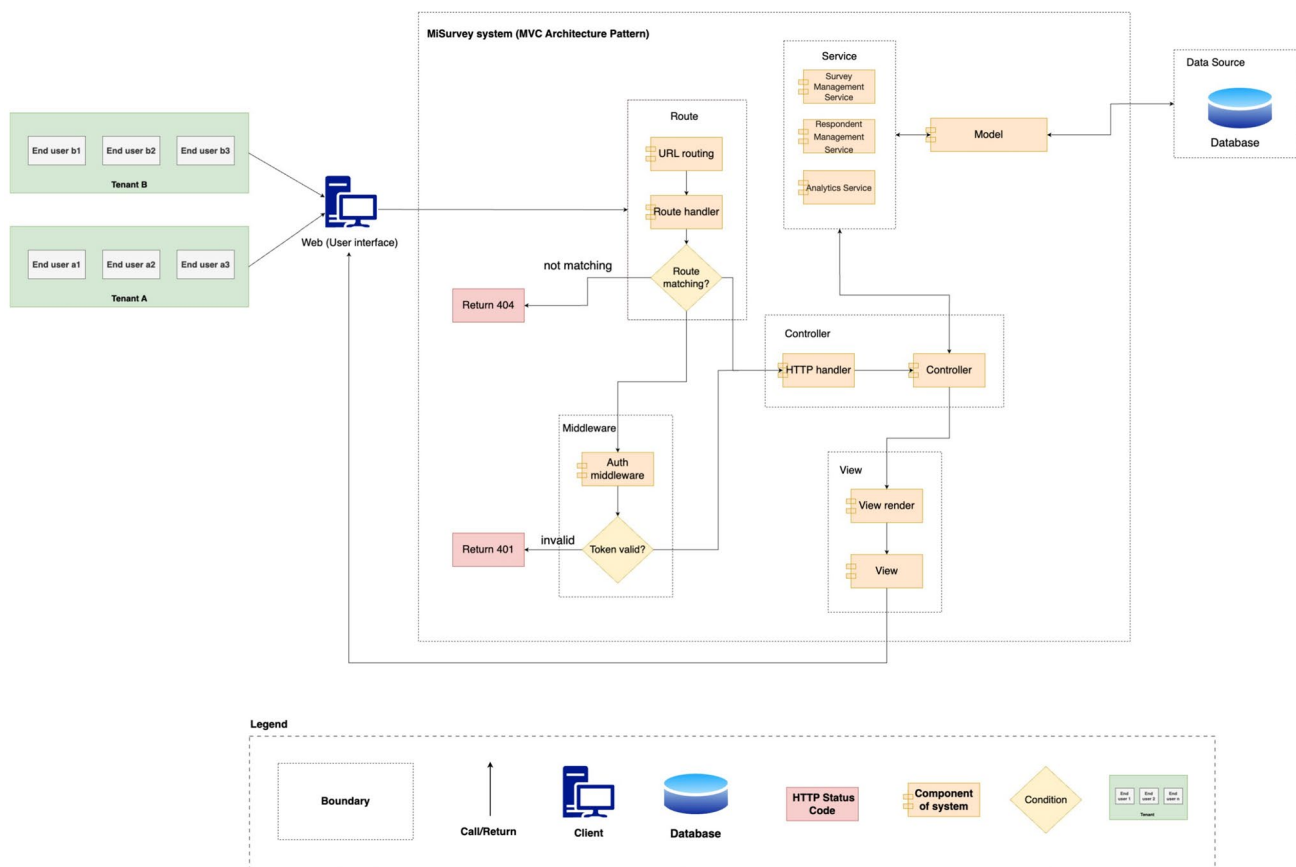


Fig. 2 Architecture of the MiSurvey System

- **User Interface (UI):** A user-friendly, web-based platform that allows users to create, manage, and analyze surveys. The UI is highly customizable, enabling tenants to tailor its appearance to match their brand identity. This approach is supported by Mietzner et al. [22], who highlighted the critical role of flexible user interfaces in multi-tenant systems.
- **Route Matching:** Route Matching serves as the initial point of interaction in our architecture's request handling process. It interprets the URL and associated parameters to determine the appropriate routing path. This is crucial for identifying which tenant's data and functionalities the request should interact with, based on URL patterns and query parameters.
- **Auth Middleware:** functions as a vital intermediary layer that handles several key tasks:
 - **Authentication and Authorization:** We employ JSON Web Tokens (JWT) within the middleware to authenticate users and authorize access to resources. JWTs store essential session data, such as user ID, tenant ID, and roles. If the token is still valid, it can be decoded to quickly retrieve user information, facilitating fast login and access to tenant-specific resources. This is particularly vital in a multi-tenant environment where maintaining data isolation and security is paramount.
 - **Session Management:** Middleware is responsible for maintaining user sessions. By checking the JWT's validity, the middleware can provide quick access to resources without requiring the user to reauthenticate, as long as the token is still valid. This improves user experience by enabling seamless and secure session continuity.
- **Controller:** The controller serves as the bridge between the Rest API and the services. It handles HTTP requests, processes incoming data, and invokes the corresponding service methods. By executing the appropriate business logic for each request, the controller plays a vital role in validating inputs, managing sessions, and delivering the correct responses to the client.
- **Model:** The model defines the database schema and manages data interactions. It employs Object-Relational Mapping (ORM) to communicate with the MariaDB database, abstracting complex database operations. The model is crucial for maintaining data integrity and enforcing constraints directly within the database. By establishing relationships and constraints in the schema, it creates a solid framework for data management, enabling efficient querying and manipulation.
- **Survey Management Service:** This service handles the creation, storage, and retrieval of surveys. It supports various types of survey questions, such as Net Promoter Score (NPS), Star Ratings, and Customer Satisfaction Score (CSAT). NPS, introduced by Reichheld [23], gauges customer loyalty and is a widely adopted metric in customer experience management. Star Ratings offer a straightforward way for customers to rate products or services, usually on a one-to-five-star scale, providing a clear quantitative assessment of satisfaction. CSAT measures the extent to which customer expectations are met or exceeded, a concept thoroughly examined by Fornell et al. [24] in relation to economic performance. Additionally, the service incorporates branching logic and extensive customization options to meet diverse survey requirements.
- **Respondent Management Service:** Handles the management of survey respondent data, including demographic details and records of their response history.
- **Analytics Service:** Delivers advanced analytical tools for processing and interpreting survey data, offering customizable reports and visualizations, and supporting integration with third-party business intelligence platforms. This component builds on the strengths of modern data analytics technologies, as highlighted by Chen et al. [25].
- **Database:** A scalable and high-performance database that stores survey data, respondent details, and tenant configurations. MariaDB is utilized for its proven reliability and efficiency in managing structured data.

Figure 2 depicts the interaction among the various components of the architecture, emphasizing the data flow and the implemented security measures. When a user interacts with the MiSurvey interface, the system processes the request through the following sequence: the Route component first identifies the request's destination based on the URL or routing parameters. Middleware may then intercept the request to perform security functions such as authentication and logging. The Route component subsequently directs the request to the appropriate controller, which processes the input and calls the relevant services to execute the desired action. During this process, the model may access the database to retrieve or update data. Finally, the controller works with the View component to construct UI elements that present the outcome to the user. Third-party libraries further enhance the system's security. Bcrypt, known for its strong password hashing capabilities, strengthens user data protection, as discussed by Provos and Mazieres [26]. Additionally, JWTs are critical for managing authentication and ensuring secure data transmission, as outlined by Jones et al. [27].

To better understand the interaction between Route Matching, Middleware, and Controller, we consider two scenarios:

- **Direct Flow from Route Matching to Controller:** For certain routes that do not require authentication or tenant context setup (e.g., login, register, forgot password), the request may bypass the middleware. In these scenarios, Route Matching can direct the request straight to the Controller to handle the logic and response generation.
- **Typical Flow Involving Middleware:** For most tenant-specific and secure resources, Route Matching identifies the correct route and passes the request through the middleware. The middleware then performs necessary checks (e.g., JWT verification) before passing the request to the Controller. The Controller processes the request using the business logic and interacts with the Model to retrieve or modify data securely within the tenant's isolated environment.

By adopting this secure multi-tenant architecture, the survey software guarantees robust data isolation, strict access control, and improved security, offering an effective solution for managing customer experience surveys across multiple businesses.

Key Design Goals

Data Isolation

The architecture guarantees strict data isolation between tenants by utilizing separate databases for each one. This approach effectively prevents unauthorized access and cross-tenant data leaks, in line with the recommendations of Chong et al. [28], who emphasize the importance of robust data isolation in multi-tenant architectures. The Advanced Encryption Standard (AES), renowned for its strength and industry-wide acceptance, secures all stored data, making it unreadable without the proper decryption key. This method safeguards sensitive customer information against breaches and unauthorized access, ensuring that each tenant's data remains confidential and protected.

Access Control

The access control in MiSurvey is carefully designed to guarantee that only authenticated and authorized users can access system resources, in accordance with the strict guidelines set by the National Institute of Standards and Technology (NIST) on access control for information systems [29]. Our architecture implements a robust Role-Based Access Control (RBAC) system, which efficiently manages

user permissions based on their designated roles within the organization. In the RBAC model, user access is not granted arbitrarily but is directly tied to their specific responsibilities and requirements:

- **Administrators** are granted comprehensive control over the platform, allowing them to configure and manage all facets of survey creation, distribution, and data analysis. Additionally, they have the authority to modify roles and permissions for other users within their tenant.
- **Supervisors**, in contrast, are granted more limited permissions, typically restricted to overseeing daily survey activities and accessing reports. These permissions are explicitly assigned by administrators, ensuring that supervisors only have access to the resources necessary for their role.
- **Respondents** are the end-users who engage with the surveys. Their access is limited to completing surveys and viewing relevant results, without any administrative privileges.

Security Measures

To uphold the highest standards of security for sensitive customer survey data within the MiSurvey platform, a carefully designed suite of security measures is integrated into the architecture. These measures are intended to protect data from unauthorized access, potential vulnerabilities, and cyber threats, ensuring that customer data is managed with the utmost confidentiality and integrity.

A key pillar of our security strategy is the encryption of sensitive data. MiSurvey implements AES with a 256-bit key, the highest industry standard, to secure data both at rest and during transmission. This ensures that all customer information stored within our databases and transmitted across networks is protected by this strong encryption protocol. Such measures not only shield data from unauthorized access but also support compliance with international data protection regulations.

To proactively mitigate security risks, the MiSurvey architecture is subjected to regular vulnerability scans. These scans are conducted using advanced automated tools that thoroughly assess our network and systems, identifying and rectifying potential security weaknesses before they can be exploited. This proactive approach ensures that any vulnerabilities are promptly addressed, maintaining the system's overall security and resilience.

MiSurvey employs a multi-layered security approach, integrating multiple defensive mechanisms such as firewalls, intrusion detection systems, and anti-malware solutions to establish a strong defense against both external and internal threats. This defense-in-depth strategy is consistent

with NIST cybersecurity guidelines, delivering a comprehensive security framework that protects every layer of the infrastructure from a wide range of attack vectors [29].

Continuous monitoring and thorough logging of all system activities are key elements of our security strategy. MiSurvey employs advanced monitoring tools that offer real-time alerts and detailed logs of system operations, which are vital for the early identification of suspicious activities that may signal a security breach. These logs are routinely reviewed by our security team, allowing for a swift response to any emerging threats.

Scalability and Performance

The architecture is designed for scalability, enabling it to support numerous tenants and concurrent users. Load balancing ensures traffic is distributed across multiple servers while auto-scaling adjusts resources dynamically according to demand. In-memory caching mechanisms like Redis [30] are used to enhance response times and reduce database load. Additionally, Nginx [31] is utilized as the web server, boosting performance by efficiently handling static content and optimizing request processing.

The MiSurvey architecture is meticulously designed for high scalability, allowing it to accommodate a large number of tenants and concurrent users. This scalability is essential for ensuring the system can seamlessly expand to meet increasing client demands without compromising performance or security.

Load Balancing and Auto-scaling: Central to our scalability strategy is the implementation of load balancing, which efficiently distributes incoming traffic and processing loads across multiple servers. This ensures that no single server is overwhelmed, maintaining high availability and consistent system performance. Moreover, MiSurvey leverages auto-scaling capabilities, which automatically adjusts resource allocation in real-time based on demand. During peak usage, extra resources are provisioned to handle the increased load, while during quieter periods, resources are scaled back to optimize cost efficiency.

In-Memory Caching with Redis: To boost performance, MiSurvey utilizes advanced in-memory caching with Redis. This technology significantly speeds up data retrieval and lowers the load on the primary databases. By storing frequently accessed data, such as user session information and popular survey results, in fast-access memory, Redis helps to reduce response times and enhances the overall user experience during interactions with the system.

Web Server Optimization with Nginx: The use of Nginx as the web server is a key strategic decision to enhance system performance. Known for its ability to manage a large number of concurrent connections with minimal memory

usage, Nginx excels at serving static content and optimizing HTTP request processing. This capability is crucial for ensuring a smooth web interface and responsive user interactions. Nginx's strong performance in managing static assets and its efficient reverse proxy functionality play a vital role in improving the overall speed and efficiency of the MiSurvey platform.

In conclusion, the proposed multi-tenant architecture provides a secure, scalable, and efficient solution for managing customer experience surveys. It successfully tackles challenges related to data isolation, access control, and security, while offering a comprehensive range of features for survey creation, distribution, and analysis.

Implementation Details

This section delves into the practical implementation of the proposed secure and scalable multi-tenant architecture for MiSurvey, detailing the technologies used, the challenges faced during development, and the solutions implemented to address them.

Deployment Setup

The MiSurvey platform is built on a solid foundation of modern web technologies. The frontend UI utilizes Angular, a popular JavaScript framework known for its capability to develop dynamic and responsive web applications [32]. With its component-based architecture and robust tooling, Angular facilitates the creation of complex user interfaces, ensuring a smooth and seamless user experience.

On the backend, Node.js, a high-performance JavaScript runtime environment, is used to manage server-side logic and API endpoints [33]. Complementing this is Express.js, a lightweight web framework for Node.js, which offers a streamlined approach to building web applications and APIs, simplifying routing and middleware integration.

To manage incoming traffic and bolster security, Nginx, a high-performance web server and reverse proxy, serves as the API Gateway. Nginx efficiently handles request routing, load balancing, and SSL/TLS termination, ensuring both optimal performance and secure communication between clients and backend services. This approach follows best practices for cloud infrastructure management, as outlined in [34].

The database layer is built on MariaDB, a robust and efficient relational database management system that is compatible with MySQL and optimized for high performance [35]. MariaDB's features, such as database-level encryption and user privilege management, are utilized to ensure data isolation and enhance security.

In addition, Fig. 3 illustrates the system allocation of the MiSurvey platform. It outlines the interaction process between a client in the user's network and a server in the server network via the Internet. The client sends a request through the Internet, and this request is navigated through a firewall before reaching the server's Rest API. At the server, the Auth Middleware takes on the task of authenticating the request before processing it. Once authenticated, the request is processed by the server, which may involve interacting with a database (MariaDB in this case) to retrieve or update information. The server uses Node.js as an environment to run the Rest API. After the request is processed, the response data is sent back through the Internet to the client.

The functional requirements of the MiSurvey system are listed in Table 1 regarding four types of actors:

- **Superadmin:** The individual or team responsible for creating and maintaining the system, typically a development team or senior-level administrator.
- **Administrator:** A representative of a business entity (company) who utilizes the system to design, manage, and oversee surveys.
- **Supervisor:** A monitoring user granted permissions by an Administrator to assist in survey management. The Supervisor helps the Administrator in managing and executing surveys, with access and customization rights potentially limited according to the Administrator's settings.

- **Customer:** The end-user who participates in surveys by receiving and submitting responses.

To evaluate the core functionalities of the MiSurvey system, we set up a test environment following the diagram in Fig. 3. This environment operated on a small scale with a limited number of tenants and users. In this environment, tenant-level data isolation was verified through schema separation in MariaDB, ensuring that each tenant's data could not be accessed by unauthorized users. Security features such as JWT-based session management, Bcrypt password hashing, and RBAC were implemented and tested to confirm that access privileges aligned with predefined roles. For scalability, we simulated multiple concurrent survey submissions using Apache JMeter with incremental load profiles. While these tests are limited in scope, they demonstrate that the architecture can support isolation, security, and scaling behaviors in practice, even under constrained conditions. Additionally, it operated stably based on key metrics such as response time and resource utilization. Overall, our initial tests indicate promising potential for the architecture. However, in order to assess its effectiveness in large-scale real-world applications, more extensive evaluations will be conducted in our future work.

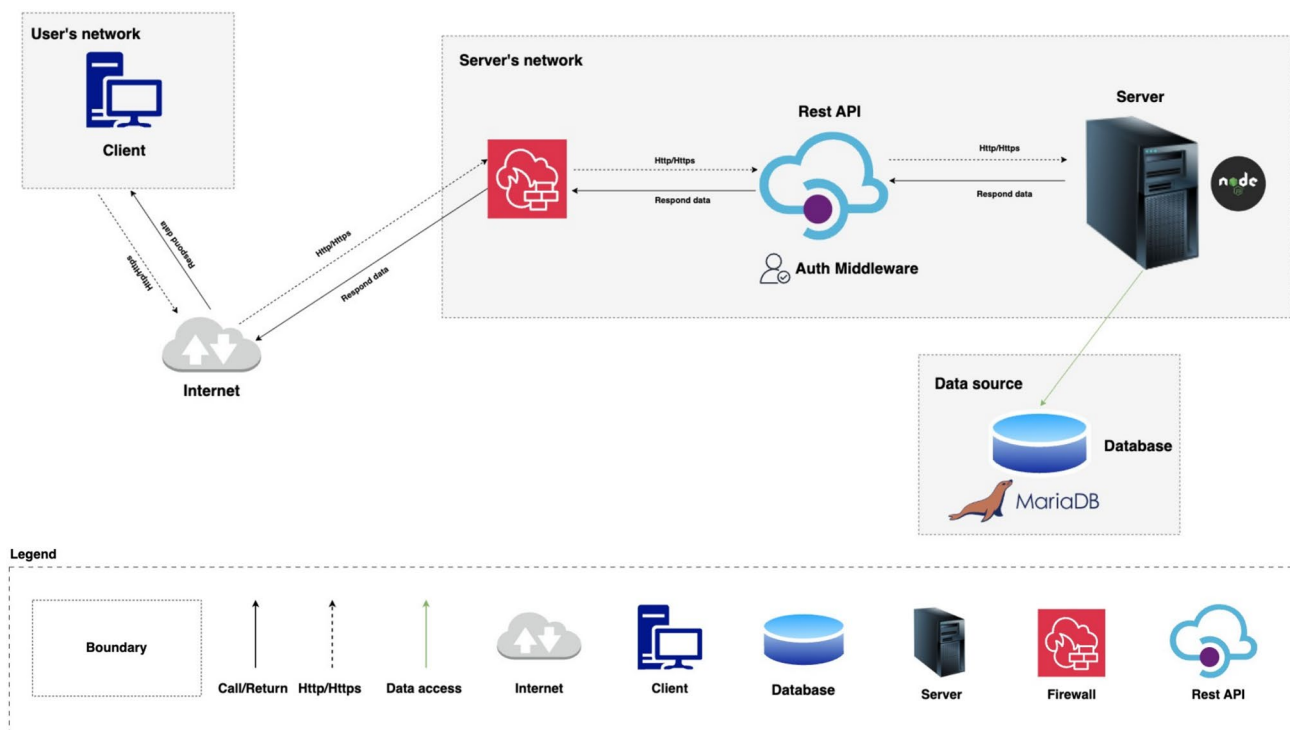


Fig. 3 Allocation Diagram of the MiSurvey System

Table 1 Functional Attributes of the MiSurvey System

Actors	Modules	Functional requirements
Superadmin (system level)	User Management	- Create user accounts - Edit user details - Delete user accounts - View users - Search users - Deactivate user accounts - Export user list
	Company Management	- Create companies - Edit company details - Delete companies - Search companies
	Question Template Management	- Create question templates - Edit question templates - Delete question templates - View question templates
	Report Management	- View activity statistics - View survey count over time - View survey type usage statistics
	User Activity Management	- View user activity logs
	Authentication	- Login - Log out - Password recovery - Sign up
	Company Profile Management	- View profile - Edit profile
	Supervisor Management	- Create supervisor accounts - Edit supervisor details - Delete supervisor accounts - View supervisors - Search supervisors - Export supervisor list
	Survey Management	- Create survey - Edit surveys - Delete surveys - Search surveys - Open / close surveys - Approve / disapprove surveys created - Invite customers to do surveys
	Report Management	- View activity statistics at the business level - View survey count over time for the business - View survey type usage statistics at the business level - View detailed survey reports - Search business-level reports
	Customer Management	- Create customers - Edit customers - Delete customers - View customers - Search customers
	User Activity Management	- View user activity logs
	Package Management	- View service packages - Upgrade service packages
	Notification Management	- Receive alerts for low ratings of surveys from customers
Customer (end-user level)	Feedback Management	- Respond to survey - Access survey via email, link, or QR code methods

Supervisors share the same functional requirements as Administrators, but their permissions are explicitly defined and assigned by the Administrator.

Challenges and Solutions

During the development of MiSurvey, several challenges were identified and effectively resolved:

Data Isolation Ensuring strict data separation between tenants was a critical objective during development. This was achieved by deploying separate databases for each tenant and enforcing robust user privilege controls within MariaDB to block unauthorized access to tenant-specific data. In a multi-tenant SaaS environment, maintaining strong data isolation and security is a fundamental necessity. Traditional data models with standard tables and relationships lack the flexibility and security required for such an environment.

To address this, MiSurvey adopts a shared database strategy, using a schema that includes a company ID column to distinguish and segregate data for different tenants. In this model, a centralized database stores all tenants' data, but each record is linked to a specific tenant via a unique CompanyID. For instance, the Company table holds general information about each tenant, such as their logo, name, and domain, while the Survey table records survey details, associating each survey with its respective company through the CompanyID field.

This design ensures that when a user from Company A accesses the system, they only interact with data tied to CompanyID = 1, remaining completely isolated from Company B's data (where CompanyID = 2). A visual representation of this database structure is shown in Fig. 4.

In this design, the CompanyID column in each table serves to isolate data for each company, ensuring that information remains segregated across tenants. This strategy effectively balances security and efficiency, offering a solid foundation for managing multi-tenant data.

A key advantage of this model is its cost efficiency. By utilizing a shared database, multiple tenants can share the same infrastructure, significantly reducing hardware and maintenance expenses. Moreover, the model facilitates seamless scaling, as adding new tenants requires minimal changes to

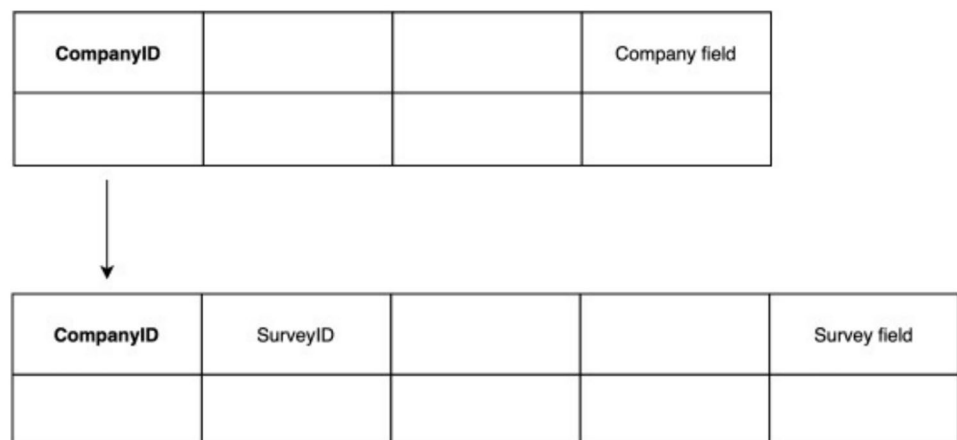
the existing database schema. Centralized management also simplifies data administration and backup processes, making overall system maintenance more straightforward.

Nevertheless, this approach also presents certain challenges. Additional security measures, such as access controls and regular audits, are required to ensure that tenant data remains properly isolated and protected. Managing tenant-specific data can become increasingly complex, especially when maintaining data consistency and integrity across shared tables. Furthermore, restoring data for an individual tenant from backups is more complicated compared to environments using separate databases.

Scalability and Performance To ensure scalability, MiSurvey adopts a combination of load balancing, database-level optimization, auto-scaling, and caching mechanisms commonly recommended in multi-tenant SaaS architectures.

- A load balancer (e.g., Nginx) distributes incoming requests across multiple servers, optimizing resource utilization and reducing the risk of bottlenecks. This design ensures that no single server becomes overloaded, thereby enhancing both reliability and responsiveness under high-traffic conditions.
- Database optimization for high workloads: Scalability at the data layer is addressed through indexing strategies and query optimization, which collectively improve query execution times and reduce latency [36]. These techniques align with established practices in cloud-native database management for SaaS systems, as noted in [34, 37].
- Elastic resource allocation: The architecture leverages auto-scaling policies, where the system can automatically scale up during peak survey campaigns (e.g., semester-end university surveys or nationwide customer campaigns) and scale down during low-traffic periods, thereby balancing cost and performance efficiency.
- In-memory caching for performance at scale: To reduce database load and improve response times during peak

Fig. 4 Utilizing CompanyID Columns for Data Partitioning



usage, MiSurvey integrates Redis-based in-memory caching for frequently accessed data [30], such as survey templates, user session data, and analytics summaries. Caching minimizes repetitive database queries, providing sub-millisecond data retrieval speeds for common operations. This is particularly effective for multi-tenant environments where multiple tenants may query similar metadata concurrently. Additionally, efficient algorithms and data structures were employed to minimize computational overhead and enable fast processing of survey responses and analytics. **Security** Protecting sensitive customer survey data is a critical priority. A multi-layered security strategy has been adopted to ensure data integrity, confidentiality, and availability. This strategy includes data encryption, rigorous input validation to prevent injection attacks, secure session management, and RBAC. To protect data at rest, MariaDB's encryption capabilities are leveraged, ensuring that sensitive survey data is encrypted within the database. Encryption keys are securely managed and rotated periodically to bolster security. Additionally, Bcrypt was used to hash passwords, providing an extra layer of security to protect against unauthorized access [26]. For secure session management, JWT tokens [27] are used. JWTs provide a compact and self-contained means of transmitting information between parties as a JSON object. Since these tokens are digitally signed, they can be verified and trusted. Upon user login, the server generates a JWT that includes user-specific information such as the user ID and roles. This token is sent to the client and stored securely. For subsequent requests, the client sends the JWT in the authorization header, and the server validates the token, ensuring it has not been tampered with or expired. JWTs also feature an expiry time, minimizing risks associated with long-lived tokens and necessitating periodic re-authentication.

Rigorous input validation is employed to prevent injection attacks. All user inputs are thoroughly checked against expected formats and sanitized to eliminate any malicious content. Techniques such as parameterized queries and prepared statements are used to defend against SQL injection, while input data is sanitized to prevent cross-site scripting (XSS) attacks by removing executable scripts. Furthermore, passwords are securely hashed using Bcrypt before being stored in the database. Bcrypt adds a unique salt to each password, protecting against rainbow table and brute-force attacks, and its adaptive design allows the iteration count to be increased as computing power increases, enhancing its resistance to attacks over time.

RBAC is a key component of our security framework, designed to ensure that users can only access the resources

and perform the actions appropriate to their assigned roles. Our system defines four primary user roles (actors): Super-admin, Administrator, Supervisor, and Business Customer. RBAC is enforced by validating the user's role and permissions before any action is performed, guaranteeing that each user interacts only with authorized functionalities. Superadmins have unrestricted access across the platform, Administrators manage resources within their specific tenant, Supervisors provide operational assistance, and Business Customers engage with surveys. By implementing RBAC at the API level, we significantly strengthen the system's security, effectively preventing unauthorized access to sensitive resources and operations.

Through the implementation of these security measures, our multi-tenant customer experience survey platform safeguards sensitive customer data and prevents unauthorized access. The combined use of data encryption, JWT-based session management, Bcrypt password hashing, and RBAC-driven access control forms a comprehensive and resilient security architecture.

Qualitative Evaluation and Comparative Analysis

While a full-scale experimental implementation and benchmarking against proprietary survey platforms (e.g., SurveyMonkey Enterprise, Google Forms Enterprise) pose challenges due to access limitations and closed-source architectures, we conducted a qualitative evaluation of MiSurvey based on its architectural features, supported by relevant academic literature and industry best practices, including recommendations from SaaS architecture guidelines (e.g., AWS SaaS Whitepapers [37]) and related research on multi-tenant systems (e.g., Mather et al. [17], Mietzner et al. [38]).

Table 2 presents a high-level comparison between MiSurvey and three representative survey management platforms: SurveyMonkey (Enterprise),¹ Google Forms (Enterprise),² and LimeSurvey (Self-hosted).³ These platforms are chosen for their popularity, cloud-based architecture, and varying degrees of enterprise-grade features. The comparison highlights five key architectural and operational criteria. A brief overview of each platform is presented below.

- SurveyMonkey Enterprise is a cloud-based survey platform designed for organizations to collect and analyze feedback at scale. It offers advanced collaboration, centralized control, and enterprise-grade security. The platform supports features like BRAC, Single Sign-On (SSO) integration, and powerful analytics. It is widely

¹ Source: <https://www.surveymonkey.com/>

² Source: <https://workspace.google.com/products/forms/>

³ Source: <https://www.limesurvey.org>

Table 2 Comparison of MiSurvey with Existing Survey Platforms

Feature	Our MiSurvey	Survey-Monkey (Enterprise)	Google Forms (Enterprise)	LimeSurvey (Self-hosted)
Data Isolation	Full tenant-level isolation with a shared database approach using separate schemas	Basic account-level or project-level separation; no strict tenant isolation	Limited; relies on Google Workspace segregation	Instance-level isolation if self-hosted per tenant
Security Features	AES-256 encryption; RBAC; JWT; Bcrypt; input validation	AES-256 encryption; RBAC; SSO; admin controls; closed-source	Encryption via Google Cloud; limited admin control granularity	Varies by setup; supports encryption and access control if configured
Scalability	Auto-scaling; load balancing; Redis caching; Nginx optimization	Scales via cloud; proprietary	Scales via Google backend; minimal tuning options	Manual scaling and performance tuning required
Customization	Per-tenant UI themes and survey templates	Moderate; limited branding; strong logic support	Minimal; fixed UI; form logic limited	High customization if self-hosted
Multi-Tenant Architecture	Native multi-tenancy with tenant-level isolation	No built-in support; relies on account separation	Workspace separation; no tenant-level enforcement	No multi-tenancy; replicate instances per tenant

used in business, education, and healthcare sectors. While robust in functionality, it does not natively support multi-tenant architecture (no public documentation or developer guide is indicating native multi-tenant support).

- Google Forms (Enterprise) is a cloud-based survey and form-building tool integrated into Google Workspace. It allows users to create forms with various question types, collect responses in real-time, and analyze data directly within Google Sheets. The enterprise version enhances collaboration and security with features like SSO, access control, and data encryption via Google Cloud. It offers limited customization of branding or layout. Google Forms is widely used due to its ease of use and integration with the Google ecosystem.
- LimeSurvey is a free and open-source survey tool designed for creating and managing online questionnaires. It offers extensive flexibility, allowing users to host the system on their own servers and customize it to meet specific data collection needs. With support for different question types, multilingual surveys, and conditional logic, it is well-suited for academic, institutional, and

enterprise use. The self-hosted version gives organizations full control over data privacy, security configurations, and scalability.

The MiSurvey architecture aligns closely with AWS SaaS best practices for secure and scalable tenant isolation [7]. It implements database schema-level isolation for each tenant, a strategy noted in literature to be more secure and manageable than row-based filtering [18, 28].

On security, compared to common configurations of public platforms, MiSurvey employs end-to-end encryption (AES-256), RBAC, JWT for secure session management, and Bcrypt for password hashing - mechanisms aligned with NIST 800–53 and contemporary SaaS security guidance [26, 27, 29].

From a scalability perspective, MiSurvey utilizes auto-scaling, load balancing, Redis caching, and Nginx optimization, practices validated in the cloud-native community [34, 36], whereas solutions like LimeSurvey require manual configuration, and Google Forms offers limited tuning capabilities.

In terms of customization, MiSurvey's architecture supports per-tenant UI themes and survey templates, which is crucial in enterprise SaaS contexts where organizations seek to maintain a consistent customer experience. Competing platforms typically limit this ability or require enterprise contracts.

Lastly, the multi-tenant architecture in MiSurvey is explicit and embedded at the design level, unlike existing solutions where tenants are simulated via separate accounts or workspaces, introducing administrative overhead and data leakage risks.

In summary, MiSurvey provides a secure, scalable, and customizable multi-tenant survey management platform that addresses several critical limitations in existing systems. While this evaluation is qualitative due to practical constraints, it is grounded in academic literature and industry standards, providing a well-substantiated comparison. Future work may include empirical performance benchmarking and user studies to validate the system's practical benefits.

Practical Scenarios

MiSurvey is a research project developed at a company to explore secure and scalable survey management in multi-tenant environments. The following scenarios illustrate its real-world potential aligned with the company's business domains and technical ecosystem. Each scenario illustrates how MiSurvey addresses the challenges typically encountered by organizations in multi-tenant environments and showcases its advantages over existing solutions.

Scenario 1: Post-Interaction Surveys for Multi-Sector Customer Experience Clients

Context: The company provides contact center and CRM platforms to clients across sectors such as logistics, banking, education, and insurance. These clients require feedback collection after service interactions to monitor agent performance and customer satisfaction.

How MiSurvey supports this: Each client organization is treated as a separate tenant in MiSurvey. Through schema-level isolation and customizable UI templates, clients can deploy post-call or post-ticket surveys aligned with their branding and workflows. MiSurvey integrates with the existing contact center platform to automate feedback requests via SMS or email, while maintaining secure data separation. This unified approach enhances both customer experience monitoring and operational efficiency.

Scenario 2: University-Wide Survey Management

Context: A university seeks to conduct course evaluation surveys across multiple departments and faculties. Each academic unit requires autonomy in designing survey templates, customizing the user interface to reflect its branding, and managing its own data access policies. At the same time, the university's central administration requires oversight and the ability to aggregate analytics for institution-wide reporting.

How MiSurvey addresses this: MiSurvey treats each faculty or department as an independent tenant. Through database schema-level isolation, each unit's data is securely separated, ensuring no leakage or unauthorized access across departments. RBAC ensures that departmental administrators manage their own surveys, while university administrators hold cross-tenant privileges. UI customization allows each faculty to brand its surveys, promoting engagement. Competing platforms such as Google Forms or SurveyMonkey lack this level of granularity in access control and branding per tenant, often requiring manual workarounds.

Scenario 3: Multi-Brand Enterprises

Context: A multinational corporation owns several consumer brands, each operating semi-independently but under a centralized IT and customer experience division. Each brand seeks to manage its own customer satisfaction surveys, with distinct branding and analytics dashboards. However, the company aims to consolidate infrastructure and reduce operational costs.

How MiSurvey addresses this: MiSurvey supports full UI customization and survey templates per tenant, enabling each brand to maintain a unique survey identity

and customer experience. The shared infrastructure model reduces maintenance overhead, while database schema-level isolation guarantees that each brand's data remains segregated and secure. Competing platforms often require separate accounts per brand, leading to redundant administration, inconsistent data handling, and increased cost. In contrast, MiSurvey provides centralized control, optimizing operational efficiency and cost-effectiveness.

These scenarios underline the practical advantages of MiSurvey's architecture in real-world deployments. The combination of multi-sector compliance, secure data isolation, customizable user interfaces, and efficient resource sharing enables MiSurvey to address the complex demands of modern multi-tenant SaaS environments more effectively than several existing survey management solutions. By accommodating diverse tenant needs within a single platform, MiSurvey enhances scalability, reduces costs, and ensures regulatory compliance - key success factors for organizations managing distributed customer experience systems.

Benefits and Limitations

The proposed architecture provides several key benefits:

- **Improved Security:** The architecture prioritizes security by implementing data isolation, access control, and a comprehensive set of security measures, ensuring the confidentiality, integrity, and availability of customer survey data.
- **Enhanced Scalability:** Leveraging load balancing and database optimization techniques, the system can efficiently accommodate a large number of tenants and users while maintaining high performance.
- **Increased Operational Efficiency:** One of the major advantages of the multi-tenant architecture is its ability to lower operational costs and streamline maintenance by sharing resources across multiple tenants. In this model, tenants utilize the same application instance, infrastructure, and database, leading to more efficient resource utilization and reduced administrative overhead. This shared approach results in significant cost savings and simplified management compared to single-tenant architectures, aligning with the economic and operational benefits of multi-tenant systems outlined by Mather et al. [17] in cloud environments.
- **Customization:** The architecture supports a high degree of customization, allowing tenants to personalize the user interface and survey templates according to their specific needs and branding. This flexibility is crucial for accommodating the diverse requirements of various

organizations. Nonetheless, certain limitations also exist:

- **Complexity:** Designing and implementing a multi-tenant architecture is inherently more challenging than a single-tenant approach, potentially resulting in higher development and maintenance costs.
- **Resource Contention:** In a multi-tenant environment, competition for shared resources among tenants can potentially impact system performance. This risk is mitigated through strategies like load balancing, resource optimization, the use of separate databases for each tenant, and the implementation of resource quotas.
- **Evaluation Experiments:** First, MiSurvey was evaluated in a small-scale test environment with a limited number of tenants and users. While this setup was sufficient to verify feasibility in terms of isolation, security enforcement, and preliminary scalability, it does not represent the full operational demands of large-scale enterprise deployments. Second, advanced testing, such as penetration testing for security validation or stress testing under thousands of concurrent users, was beyond the scope of this prototype implementation.

Conclusions

This secure and scalable multi-tenant architecture for customer experience survey software greatly improves the efficiency and security of managing surveys across multiple organizations. By leveraging a multi-tenant SaaS model, our application ensures robust data isolation and stringent user access control, effectively addressing key challenges in survey management. Additionally, the architecture streamlines operations by automating processes, minimizing manual intervention and errors, and ultimately enhancing the survey lifecycle and boosting user productivity.

The system ensures seamless data segregation, keeping each tenant's data isolated and secure. Robust security practices, including encryption, secure communication protocols, and regular audits, protect sensitive survey data from breaches and unauthorized access. Moreover, the architecture's scalability and configurability allow it to adapt to diverse business requirements, efficiently supporting multiple tenants on a shared infrastructure. This flexibility not only maintains high performance and reliability but also helps lower operational costs.

In summary, our proposed secure and scalable multi-tenant architecture for customer experience survey software addresses businesses' current demands for efficient and secure survey management while establishing a strong foundation for future scalability and security enhancements. This solution has the potential to revolutionize how businesses

manage customer feedback, reducing costs and boosting overall productivity in a secure and reliable environment.

Future research will focus on developing more advanced security protocols and scalability solutions to further optimize multi-tenant environments. In addition, large-scale experiments will be conducted in diverse workloads, performing deeper security and compliance audits, and benchmarking MiSurvey against existing platforms in controlled enterprise environments.

Acknowledgements No external funding was received for this work.

Data Availability The authors declare that data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

Declarations

Conflict of Interest The authors declare that there is no Conflict of interest regarding the publication of this paper.

Ethical Approval This study does not violate and does not involve moral and ethical statement.

Consent for Publication The authors were aware of the publication of the paper and agreed to its publication.

References

1. Ivanovska LP, Miteva A, Lozanoska A. The challenges of touchpoint management of telecommunications companies with empirical analysis for the republic of north macedonia. *Economic Development/Ekonomiski Razvoj*. 2022;24(5).
2. Halvorsrud R, Kvale K, Følstad A. Improving service quality through customer journey analysis. *J Serv Theory Pract*. 2016;26(6):840–67.
3. Deloitte: 2021 Global Marketing Trends. <https://www2.deloitte.com/xe/en/pages/strategy-operations/articles/2021-global-marketing-trends.html>. Accessed: 12 April 2025 (2021)
4. Denavi HD, Emadi SH, Manshadi M. The effect of customer relationship management on customer satisfaction and competitive advantage. *International Journal of Engineering and Technology*. 2018;10:987–94.
5. Haryandika DM, Santra IK. The effect of customer relationship management on customer satisfaction and customer loyalty. *Indonesian Journal of Business and Entrepreneurship (IJBE)*. 2021;7(2):139–139.
6. Armbrust M, Fox A, Griffith R, Joseph AD, Katz R, Konwinski A, et al. A view of cloud computing. *Commun ACM*. 2010;53(4):50–8.
7. AWS: SaaS Tenant Isolation Strategies: Isolating Resources in a Multi-Tenant Environment. <https://docs.aws.amazon.com/whitepapers/latest/saas-tenant-isolation-strategies/saas-tenant-isolation-strategies.html>. Accessed: 12 April 2025 (2020)
8. Nguyen Q-D, Pham V-N, Dang T-N, Phan T-N. A secure multi-tenant architecture for customer experience survey software. In: *International Conference on Future Data and Security Engineering*, pp. 2024:48–61
9. Bezemer C-P, Zaidman A. Multi-tenant saas applications: maintenance dream or nightmare? In: *Proceedings of the Joint Ercim*

- Workshop on Software Evolution (evol) and International Workshop on Principles of Software Evolution (iwps), 2010; 88–92
10. Pinto VHSC, Oliveira RR, Vilela RF, Souza SR. Evaluating the user acceptance testing for multi-tenant cloud applications. In: Proceedings of the 8th International Conference on Cloud Computing and Services Science, 1, 2018; 47–56
 11. Maenhaut P-J, Moens H, Ongenaes V, De Turck F. Migrating legacy software to the cloud: approach and verification by means of two medical software use cases. *Software: Practice and Experience* 2016;46(1), 31–54
 12. Pinto VH, Luz HJ, Oliveira RR, Souza PS, Souza SR. A systematic mapping study on the multi-tenant architecture of saas systems. In: 2016 International Conferences on Software Engineering and Knowledge Engineering, 2016; 396–401
 13. Kabbedijk J, Bezemer C-P, Jansen S, Zaidman A. Defining multi-tenancy: a systematic mapping study on the academic and the industrial perspective. *J Syst Softw.* 2015;100:139–48.
 14. Reddy AY, Kumar MU. Design and development of multi-tenancy security issues in cloud computing. *International Journal of Multidisciplinary Research and Growth Evaluation.* 2021;2:341–8.
 15. Ochei LC, Petrovski A, Bass JM. Evaluating degrees of tenant isolation in multitenancy patterns: A case study of cloud-hosted version control system (vcs). In: 2015 International Conference on Information Society (i-Society), 2015; 59–66
 16. Decat M, Bogaerts J, Lagaisse B, Joosen W. Amusa: middleware for efficient access control management of multi-tenant saas applications. In: Proceedings of the 30th Annual ACM Symposium on Applied Computing, 2015; 2141–2148
 17. Mather T, Kumaraswamy S, Latif S. *Cloud Security and Privacy: an Enterprise Perspective on Risks and Compliance.* California: O'Reilly Media Inc; 2009.
 18. Aulbach S, Grust T, Jacobs D, Kemper A, Rittinger J. Multi-tenant databases for software as a service: schema-mapping techniques. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, 2008; 1195–1206
 19. Samrajesh M, Gopalan N. Towards multivariable architecture for saas multi-tenant applications. *International Journal of Software Engineering and Its Applications.* 2016;10(4):13–26.
 20. Cao Y, Lung C-H, Ajila SA. Constraint-based multi-tenant saas deployment using feature modeling and xml filtering techniques. In: 2015 IEEE 39th Annual Computer Software and Applications Conference, vol. 3, 2015; 454–459
 21. Makki M, Van Landuyt D, Walraven S, Joosen W. Scalable and manageable customization of workflows in multi-tenant saas offerings. In: Proceedings of the 31st Annual Acm Symposium on Applied Computing, 2016; 432–439
 22. Mietzner R, Unger T, Titze R, Leymann F. 2009; Combining different multi-tenancy patterns in service-oriented applications. In: 2009 IEEE International Enterprise Distributed Object Computing Conference, 131–140
 23. Reichheld FF. The one number you need to grow. *Harv Bus Rev.* 2003;81(12):46–55.
 24. Fornell C, Johnson MD, Anderson EW, Cha J, Bryant BE. The american customer satisfaction index: nature, purpose, and findings. *J Mark.* 1996;60(4):7–18.
 25. Chen H, Chiang RH, Storey VC. Business intelligence and analytics: From big data to big impact. *MIS quarterly,* 2012; 1165–1188
 26. Provos N, Mazieres D. A future-adaptable password scheme. In: *USENIX Annual Technical Conference, FREENIX Track,* vol. 1999, 1999; 81–91
 27. Jones M, Bradley J, Sakimura N. Json web token (jwt). Technical report, IETF. RFC 7519 2015;
 28. Chong F, Carraro G, Wolter R. Multi-tenant data architecture. MSDN Library, Microsoft Corporation, 2006;14–30
 29. Force JT, Initiative T. Security and privacy controls for federal information systems and organizations. *NIST Spec Publ.* 2013;800(53):8–13.
 30. Priovolos T, Maroulis S, Kalogeraki V. Escape: Elastic caching for big data systems. In: 2019 38th Symposium on Reliable Distributed Systems (SRDS), 2019; 93–9309
 31. Chi X, Liu B, Niu Q, Wu Q. Web load balance and cache optimization design based nginx under high-concurrency environment. In: 2012 Third International Conference on Digital Manufacturing & Automation, 2012; 1029–1032
 32. Seshadri S. *Angular: Up and Running: Learning Angular.* California: Step by Step. O'Reilly Media Inc; 2018.
 33. Osmani A. *Learning JavaScript Design Patterns: A JavaScript and React Developer's Guide.* California: O'Reilly Media Inc; 2023.
 34. Buyya R, Broberg J, Goscinski A. *Cloud Computing: Principles and Paradigms.* 1st ed. New York: Wiley; 2011.
 35. Karwin B. *SQL Antipatterns: Avoiding the Pitfalls of Database Programming.* 1st ed. Texas: Pragmatic Programmers; 2017.
 36. Maesaroh S, Gunawan H, Lestari A, Tsaure MSA, Fauji M. Query optimization in mysql database using index. *International Journal of Cyber and IT Service Management.* 2022;2(2):104–10.
 37. Tod Golding - AWS SaaS Factory: SaaS Architecture Fundamentals: AWS Whitepaper. <https://docs.aws.amazon.com/whitepaper/latest/saas-architecture-fundamentals/saas-architecture-fundamentals.html>. Accessed: 27 July 2025 (2022)
 38. Mietzner R, Leymann F, Papazoglou MP. Defining composite configurable saas application packages using sca, variability descriptors and multi-tenancy patterns. In: 2008 Third International Conference on Internet and Web Applications and Services, 2008; 156–161

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.